
GIT, GITHUB

14기 부회장 설수현

PART1. GIT

Git은 코드의 변경 이력을 기록하고 관리하는 분산 버전 관리 시스템(VCS, Version Control System).

파일이 언제, 어떻게 수정되었는지 추적할 수 있으며, 필요할 경우 이전 버전으로 되돌릴 수 있다.
또한 여러 사람이 각자의 작업을 독립적으로 진행한 뒤 변경사항을 병합할 수 있어 협업에 유용.

- Git의 주요 특징

- 빠른 속도와 높은 성능
- 효율적인 버전 관리
- 폭넓은 활용과 협업
- 무료 오픈소스



02. GIT과 GITHUB의 차이

Git

내 컴퓨터에서 코드의 버전을 관리하는 도구

- 파일의 변경 이력을 기록하고 이전 상태로 되돌릴 수 있음
- add, commit, branch 등을 이용해 작업 내용을 관리
- 인터넷 연결 없이도 사용할 수 있음

GitHub

Git으로 관리하는 프로젝트를 온라인에 저장하고 공유하는 플랫폼

- 원격 저장소(Repository)를 통해 코드를 백업·공유
- 여러 사람이 Fork, Pull Request 등을 이용해 협업 가능
- Git을 기반으로 한 대표적인 호스팅 서비스



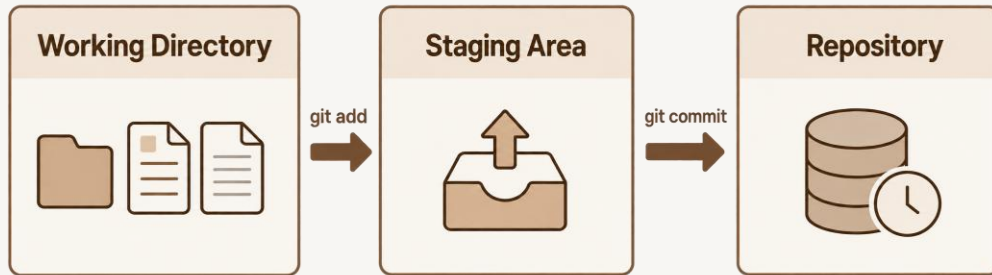
쉽게 정리하면

Git = 버전 관리 도구

GitHub = Git 저장소를 온라인에서 공유·협업하는 공간

03. GIT의 저장공간

작업한 파일을 바로 저장하는 것이 아니라, 필요한 변경사항을 선택해 Stage에 올린 뒤 Commit으로 기록하는 구조



1. Working Directory

- 실제로 파일을 작성·수정하는 작업 공간
- 아직 Git에 기록되지 않은 변경사항이 존재하는 곳

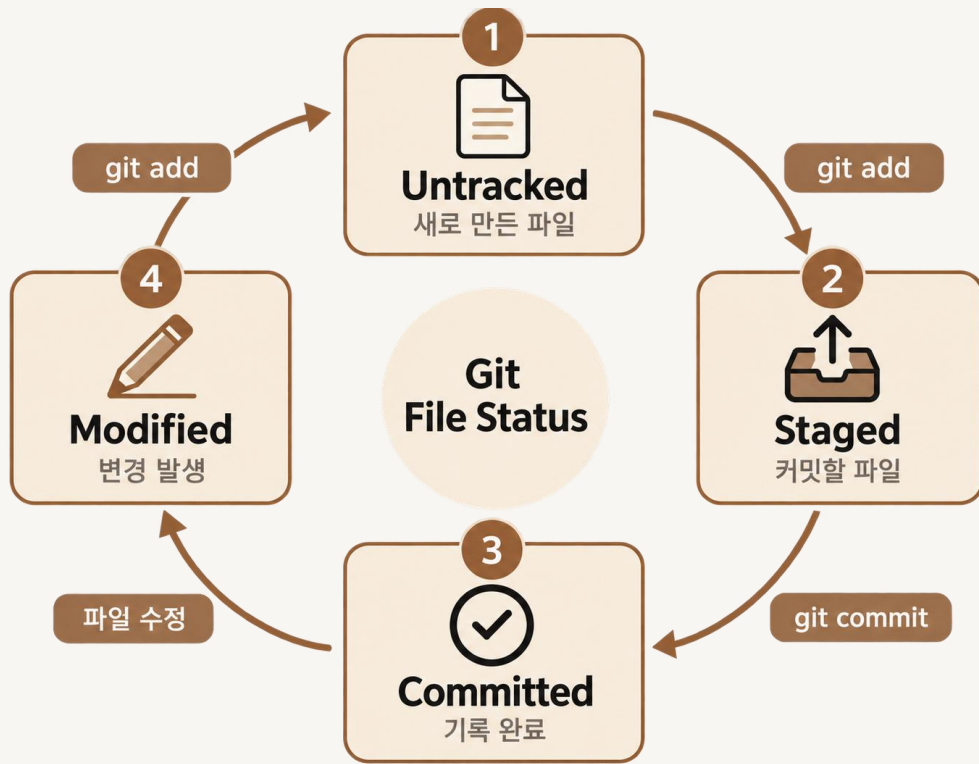
2. Staging Area

- 다음 Commit에 포함할 파일을 임시로 올려두는 공간
- `git add`를 통해 파일을 Stage에 추가

3. Repository

- 변경사항을 Commit 단위로 저장하고 기록하는 공간
- `git commit`을 통해 현재 상태를 하나의 버전으로 저장

04. GIT 파일상태



1. Untracked

- Git이 아직 추적하지 않는 파일
- Ex) 새로 생성한 파일

2. Staged

- `git add`를 통해 다음 Commit에 포함하도록 올려둔 상태
- Git이 해당 파일을 추적하기 시작함

3. Committed / Unmodified

- `git commit`을 통해 변경사항이 저장된 상태
- 이후 수정하기 전까지 변경사항이 없음

4. Modified / Unstaged

- 이미 추적 중인 파일을 다시 수정한 상태
- 수정된 내용을 Commit하려면 다시 `git add`가 필요함

05. BRANCH

Branch는 하나의 저장소에서 독립적으로 작업할 수 있도록 만든 별도의 작업 흐름.

- 메인 작업을 건드리지 않고 새로운 기능이나 작업을 따로 진행할 수 있음
- 각 Branch에서 작업한 뒤 필요할 경우 다시 병합(Merge) 가능
- 여러 사람이 동시에 작업할 때 서로의 작업이 섞이는 것을 줄일 수 있음

- 명령어

1. 현재 branch 목록 확인

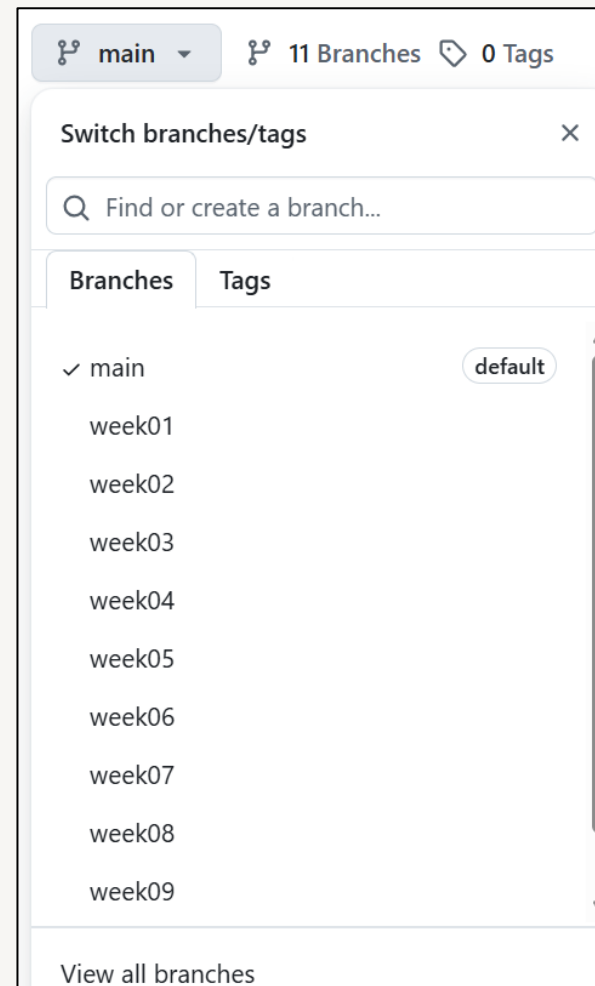
`git branch -v`

2. 해당 branch로 이동

`git checkout week01`

3. 새 branch 생성 후 바로 이동

`git checkout -b week01`



06. FORK & CLONE

Fork

다른 사람의 Repository를 내 GitHub 계정으로 복사하는 기능

- 원본 Repository를 내 GitHub 계정에 복제
- 원본을 직접 수정하지 않고 내 Repository에서 자유롭게 작업

Clone

GitHub에 있는 Repository를 내 컴퓨터로 내려받는 명령어

- 원격 Repository의 파일과 Git 이력을 로컬 컴퓨터로 복사
- 로컬에서 직접 코드 수정 및 Git 작업 가능
- Fork한 Repository도 Clone해서 작업할 수 있음

Create a new fork

A *fork* is a copy of a repository. Forking a repository allows you to freely experiment with changes without affecting the original project. [View existing forks.](#)

Required fields are marked with an asterisk (*).

Owner *

 dnamu8184 ▾

Repository name *

14th_Seminar

✔ 14th_Seminar is available.

By default, forks are named the same as their upstream repository. You can customize the name to distinguish it further.

Description

0 / 350 characters

☒ Copy the `main` branch only

Contribute back to konkuk-kuggle/14th_Seminar by adding your own branch. [Learn more.](#)

 You are creating a fork in your personal account.

Create fork

07. ADD & COMMIT

git add

변경된 파일을 Staging Area에 올리는 명령어

- 새로 만든 파일을 Git이 추적하도록 등록
- 다음 Commit에 포함할 파일을 선택
- 수정된 파일은 다시 git add 해야 최신 변경사항이 Stage에 반영됨

git commit

Staging Area에 올라간 변경사항을 하나의 버전으로 기록하는 명령어

- Commit 단위로 변경 이력을 저장
- Commit message를 통해 어떤 작업을 했는지 기록

자주 쓰이는 명령어

1. git add 파일명

특정 파일 Stage에 추가

2. git add .

모든 변경사항 Stage에 추가

3. git commit -m "message"

Staged 변경사항을 Commit

08. PUSH& PULL

Git push

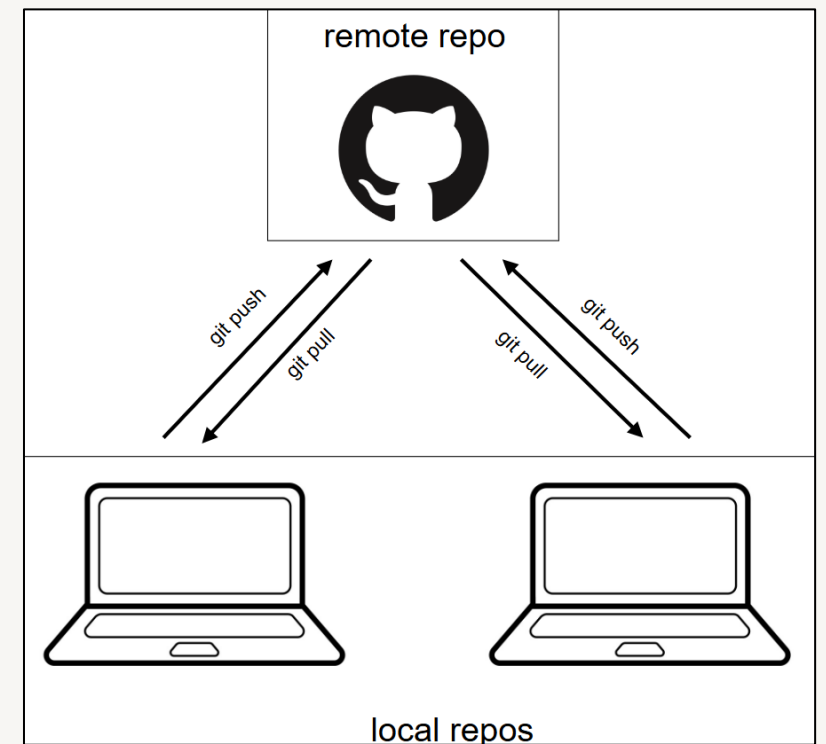
로컬 Repository의 Commit 내용을 GitHub 원격 Repository에 업로드하는 명령어

- 로컬에서 add → commit한 내용을 GitHub에 반영
- 특정 Branch의 작업 내용을 원격 Repository로 전송
- 원격 Repository와 연결된 상태에서 사용

Git pull

GitHub 원격 Repository의 변경 내용을 로컬 Repository로 가져오는 명령어

- 원격 Repository의 최신 Commit 내용을 로컬에 반영
- 특정 Branch의 변경 내용을 로컬 Branch로 가져옴
- 원격 변경사항을 가져온 뒤 현재 로컬 Branch에 병합
- GitHub와 연결된 로컬 Repository에서 사용



09. PULL REQUEST

내가 작업한 변경사항을 원본 Repository에 반영해 달라고 요청하는 기능

- 개인 Repository / Branch에서 작업한 내용을 원본 Repository에 제안
- 변경사항을 확인한 뒤 Merge 가능
- 협업 시 바로 main을 수정하지 않고 검토 후 반영할 수 있음

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#). [Learn more about diff comparisons here.](#)

 base repository: konkuk-kuggle/13th_Seminar ▼

base: week01 ▼

...

head repository: dnamu8184/13th_Seminar ▼

compare: week01 ▼

✖ Can't automatically merge. Don't worry, you can still create the pull request.



Add a title *

week01_설수현

Add a description

WritePreview

H B I                             

Add your description here...

Reviewers

Suggestions

 jaehwan50

Assignees

No one—assign

Labels

PART2. GIT 설치(윈도우)

설치 링크: <https://git-scm.com/>



별도의 설정 변경 없이 기본값(Default)으로 설치를 진행

PART2. GIT 설치(MAC)

1. Git 설치 여부 확인

Terminal에서 `git --version`

버전 정보가 출력되면 Git이 이미 설치되어 있는 상태.

2. Git 설치

Git이 설치되어 있지 않다면 Homebrew를 이용해 설치.

`brew install git`

※ Xcode가 설치되어 있다면 별도의 Git 설치 필요X.

3. 설치 확인

설치 후 다시 다음 명령어를 입력해 정상 설치 여부를 확인.

`git --version`

PART3. 과제

1. KUGGLE 세미나 Repository를 본인 GitHub 계정으로 Fork
2. 과제 파일을 본인 Repository의 week01 Branch에 업로드
3. 업로드한 내용을 KUGGLE 14th Seminar의 week01 Branch로 Pull Request 제출

Github 링크: [konkuk-kuggle/14th_Seminar](https://github.com/konkuk-kuggle/14th_Seminar) , title: week01_설수현

바탕화면에서 폴더 만들고 과제 파일 넣기 -> git clone -> git checkout week01 -> git add -> commit -> push -> pull request